

Voice-Cloning Streaming TTS: Step-by-Step Build Plan

From Speechify Clone to Real-Time Voice-Cloning TTS Engine

Phase 0 — Orientation and Baseline

Phase 1 — Audio Codec Foundation

Phase 2 — Voice-Conditioned Generation (the cloning core)

Phase 3 — Streaming, Chunked Generation (the hard, differentiating part)

Phase 4 — Latency Optimization

Phase 5 — Evaluation Harness

Phase 6 — Robustness and Failure-Mode Hardening

Phase 7 — Integration into the Existing App

Phase 8 — Frontend/UI Integration

Phase 9 — Monitoring and Observability

Phase 10 — Deployment and Final Packaging

From Speechify Clone to Real-Time Voice-Cloning TTS Engine

A sequential, prompt-by-prompt implementation plan

How to use this document: Copy one “PROMPT” block at a time into your IDE’s AI coding assistant (Claude Code, Cursor, Copilot Chat, whatever you’re using). Run it, do the verification steps yourself, confirm the acceptance checklist passes, and only then move to the next prompt. Every prompt starts by re-inspecting the codebase and re-checking the previous step’s acceptance criteria — never assume state carries over correctly.

Before Prompt 1, create an empty file at the root of your repo called `PROJECT_STATE.md`. This is the persistent memory of the project across prompts. Every prompt below ends by telling the assistant exactly what to append to it. Never let the assistant delete history from this file — only append.

Phase 0 — Orientation and Baseline

PROMPT 0: Audit the existing Speechify clone

Before writing any code, do a full audit of this repository and produce a written summary.

1. Inspect the project structure (list all directories and key files).
2. Identify: the language/framework used (backend and frontend if separate),
the current TTS approach (which model/library/API is used to convert text to speech), how audio is currently generated and returned to the user (batch file? streaming? what format?), any existing tests and how to run

- them, and any existing voice/speaker customization features.
3. Identify what is NOT present: neural audio codecs, voice cloning from
reference audio, streaming/chunked generation, latency instrumentation,
speaker-similarity evaluation.
 4. Run the existing test suite and report pass/fail status and coverage,
if measurable.
 5. Create PROJECT_STATE.md at the repo root with sections:
Architecture Snapshot (date)
Current TTS Approach
Existing Test Status
Gaps Relative to Target Project
Decisions Log (empty, to be filled in later steps)
Fill in everything you found. Do not modify any application code in this step.

What we're building and why: Nothing yet — this step exists because every later prompt depends on an accurate map of what's already there. Skipping it is the single most common way these multi-step builds go off the rails: the assistant starts assuming files or functions exist that don't.

What to inspect: The whole repo tree, package.json/requirements.txt/pyproject.toml (whichever applies), the entry point of the TTS pipeline, and the test runner config.

What you should learn: Get honest with yourself about what your "Speechify clone" currently does. If it's just calling a third-party TTS API (Google Cloud TTS, AWS Polly, etc.) rather than running a model, say so plainly in PROJECT_STATE.md — it changes how much of the rest of this plan is "add a new capability" versus "replace the core."

Testing/verification: Confirm PROJECT_STATE.md exists and accurately reflects reality — read it yourself and correct anything wrong before continuing.

Acceptance checklist: - [] PROJECT_STATE.md exists with all five sections filled in - [] You have personally verified the summary is accurate - [] Existing test suite runs (even if some tests fail — note which ones) - [] No application code was changed

Phase 1 — Audio Codec Foundation

PROMPT 1: Add neural audio codec encode/decode as an isolated module

Read PROJECT_STATE.md for context on the existing architecture.
Then:

1. Add a new, isolated module (e.g. `audio\_codec/` or `services/codec/`) that has no dependencies on the existing TTS pipeline yet.
2. Integrate a pretrained neural audio codec (use Meta's EnCodec via the `encodec` or `transformers` Python package — pick whichever has better maintained docs — or Descript's DAC if you prefer). Do NOT train a codec from scratch.
3. Implement two functions: `encode\_audio(waveform, sample\_rate) -> tokens` and `decode\_tokens(tokens) -> waveform`.

4. Write a script that takes a real audio file (record or download a ~10 second clean speech sample), round-trips it through encode then decode, and saves the output next to the input.
5. Write a unit test that asserts: output sample rate matches input, output duration is within 5% of input duration, and the round-tripped audio's RMS energy is within a reasonable range of the original (not silence, not clipped noise).
6. Do NOT touch the existing TTS pipeline in this step.

Why this matters: This is the representation everything else builds on. ElevenLabs- and VALL-E-style architectures treat speech as a sequence of discrete tokens instead of raw waveforms or spectrograms — that's what makes cheap voice conditioning and autoregressive streaming possible later. If you don't understand what a codec token actually encodes, the rest of the project will feel like magic instead of engineering.

What to inspect: Nothing in the old pipeline yet — this module is deliberately decoupled so you can develop and test it without risking regressions elsewhere.

What to learn: Listen to the round-tripped audio yourself. Notice what's preserved (words, rough voice identity) and what's degraded (some timbre, background noise) at the default codebook/bandwidth setting. Try a lower and higher bandwidth setting and listen to the difference — this is your first real encounter with the quality/compression tradeoff you'll be discussing in interviews.

Testing: Run the unit test. Also do a manual listening test — this is not optional, automated audio metrics alone will not catch a codec silently producing garbage that technically passes numeric assertions.

Edge cases to check: Very short audio (<1 second), audio with a different native sample rate than the codec expects (resampling correctness), silent/near-silent input, stereo vs. mono input.

Docs to update: Add a `audio_codec/README.md` explaining what a neural codec is in your own words, which model/checkpoint you used, and the bandwidth setting you chose and why.

Update `PROJECT_STATE.md`:

```
## Decisions Log
- [date] Chose <EnCodec/DAC>, checkpoint <name>, bandwidth <X kbps>
  because <reason>.
## Codec Module
- Location: audio_codec/
- Verified working via: <test file>
- Known limitations: <e.g. degrades on noisy input>
```

Acceptance checklist: - [] Round-trip unit test passes - [] You've personally listened to at least 3 round-tripped samples - [] Edge cases above are tested and behavior is documented (even if it's "fails ungracefully — will fix in Phase 6 hardening") - [] No regressions in the existing test suite from Prompt 0 - [] `PROJECT_STATE.md` updated

Phase 2 — Voice-Conditioned Generation (the cloning core)

PROMPT 2: Integrate a pretrained voice-cloning TTS model

Read PROJECT_STATE.md, especially the Codec Module section. Then:

1. In a new module (e.g. ``voice_clone/``), integrate a pretrained open voice-cloning TTS model that supports few-shot/zero-shot cloning from a short reference audio clip – use Bark, XTTS (Coqui), or a VALL-E reimplementation, whichever has the most usable Python API. Do not train a model from scratch.
2. Implement ``clone_and_synthesize(text: str, reference_audio_path: str) -> waveform``. This should NOT be streaming yet – full-file synthesis only in this step.
3. Write an integration test: given a fixed reference speaker sample and a fixed sentence, generate audio twice and confirm the pipeline runs without error both times and produces non-trivial audio (same length bounds and RMS checks as Prompt 1).
4. Manually evaluate speaker similarity: install a pretrained speaker embedding model (e.g. ``speechbrain``'s ECAPA-TDNN, or `resemblyzer`), embed both the reference clip and the generated clip, and compute cosine similarity. Print the score.
5. Do NOT integrate this into the existing app UI/API yet – this stays a standalone module callable from a script.

Why this matters: This is the actual “ElevenLabs feature” — most tutorials build TTS for a single fixed voice; the hard, differentiating problem is generalizing to an arbitrary new voice from a few seconds of audio it has never seen. This is also the step where you should genuinely study *why* this works: the reference audio is used as a conditioning prompt so the model’s learned speech distribution gets steered toward that speaker’s timbre and prosody without any gradient updates or fine-tuning.

What to inspect: The codec module’s output format from Phase 1, since some of these models expect codec tokens directly as conditioning; others want raw audio. Check the actual API surface of whichever library you chose.

What to learn: Read (at least skim) the VALL-E paper or the XTTS technical writeup, whichever matches the model you picked. Understand specifically how “speaker prompt” conditioning is implemented in that architecture — you’ll be asked this in an interview if you claim this project.

Testing: Integration test above, plus manual listening comparing the cloned output against the reference voice.

Verification of correctness: The speaker-similarity cosine score should be meaningfully higher when comparing generated audio to the *correct* reference speaker than to a *different, randomly chosen* speaker’s audio. Test both and confirm the gap exists — this rules out a broken conditioning path silently producing a generic voice.

Edge cases: Very short reference clips (2–3 seconds) vs. longer ones (15+ seconds) — does similarity improve with more reference audio? Reference audio with background noise. Text containing numbers, abbreviations, or punctuation the model might mishandle.

Docs to update: `voice_clone/README.md` — which model, why, how conditioning works, and your speaker-similarity numbers for at least 3 different reference voices.

Update `PROJECT_STATE.md`:

```
## Decisions Log
- [date] Chose <model> for voice cloning because <reason>.
## Voice Cloning Module
- Location: voice_clone/
- Speaker similarity (correct pair vs mismatched pair): <numbers>
- Known limitations: <e.g. struggles with very short reference clips>
```

Acceptance checklist: - [] Integration test passes reliably (run it 3 times) - [] Speaker similarity is measurably higher for correct vs. mismatched reference audio - [] You can explain, in your own words, how the reference audio steers generation - [] No regressions from Phase 0/1 - [] `PROJECT_STATE.md` updated

Phase 3 — Streaming, Chunked Generation (the hard, differentiating part)

PROMPT 3: Convert full-file synthesis into streaming, chunked generation

Read `PROJECT_STATE.md`, especially the Voice Cloning Module section. Then:

1. Modify (or wrap, if the underlying library resists modification) ``clone_and_synthesize`` into a generator/streaming version:
``stream_synthesize(text, reference_audio_path) -> yields audio chunks``
2. Implement chunk-boundary smoothing: apply a short crossfade (e.g. 10–20ms linear or raised-cosine fade) between consecutive decoded chunks so there is no audible click or pop at the seams. Write this as its own pure function so it's independently testable.
3. Instrument the pipeline to record: time from request start to first audio chunk emitted ("time to first audio", TTFA), and total generation time divided by output audio duration ("real-time factor", RTF).
4. Write a test script that runs `stream_synthesize` on 5 different sentences of varying length and logs TTFA and RTF for each, saving results to ``benchmarks/streaming_baseline.json``.
5. Write a unit test for the crossfade function specifically: construct two synthetic sine-wave chunks with a deliberate phase/amplitude discontinuity, apply the crossfade, and assert the discontinuity at the seam is reduced below a threshold you define.

Why this matters: This is the single most technically interesting and most rarely-implemented-correctly part of the whole project. Full-file generation is what every “TTS clone” tutorial does. Chunked, low-latency streaming with clean audio boundaries is the actual systems problem that separates a demo from something that could sit inside a real conversational product. If you can only explain one part of this project in depth during an interview, it should be this one.

What to inspect: Whether the underlying model library exposes any native streaming/incremental decoding hooks, or whether you need to synthesize in fixed-size overlapping windows and stitch them yourself (the more common and more educational case).

What to learn: Why naive chunking (just cutting the output into pieces after the fact) doesn’t reduce latency at all — true latency reduction requires the *generation* itself to be incremental, not just the playback. Understand the difference between those two and be explicit in your documentation about which one you implemented, especially if your chosen model doesn’t support true incremental generation and you had to approximate it.

Testing: Run the benchmark script. Look at the actual TTFA numbers — if they’re not meaningfully better than full-file synthesis, that’s a real finding, not a failure; document it honestly and treat the next prompt (optimization) as the place to fix it.

Edge cases: Very short text (a single word) — does chunking even make sense/trigger? Text that naturally has no good chunk boundary (a single very long unbroken clause). Silence-only chunks (does the crossfade break on near-zero-amplitude audio)?

Docs to update: voice_clone/STREAMING.md explaining the chunking strategy, the crossfade approach, and your baseline TTFA/RTF numbers with methodology (hardware used, sentence lengths tested).

Update PROJECT_STATE.md:

```
## Decisions Log
- [date] Chunk size chosen: <N tokens/ms> because <reason>.
- [date] True incremental generation: <yes/approximated> –
  <explanation>
## Streaming Module
- Baseline TTFA: <value>, Baseline RTF: <value> (see
  benchmarks/streaming_baseline.json)
- Known limitations: <...>
```

Acceptance checklist: - [] Crossfade unit test passes with a defined, justified threshold - [] Baseline benchmark file exists with numbers for 5+ sentences - [] You can explain the difference between true incremental generation and post-hoc chunking, and state honestly which one you built - [] No regressions from prior phases - [] PROJECT_STATE.md updated

Phase 4 — Latency Optimization

PROMPT 4: Reduce time-to-first-audio and real-time factor

Read PROJECT_STATE.md, especially the baseline TTFA/RTF numbers from Phase 3. Then, pick ONE concrete optimization to implement (don't do all of these at once):

Option A: Reduce chunk size to emit audio sooner, and measure

whether
audio quality/naturalness at the (now more frequent) seams is still acceptable.
Option B: If your model runs on GPU, ensure it's actually using it (check device placement), and add mixed-precision inference (fp16/bf16) if not already enabled.
Option C: Cache/reuse the reference-speaker conditioning computation across multiple requests for the same speaker instead of recomputing it every call.

1. Before changing anything, re-run the Phase 3 benchmark script and record the "before" numbers explicitly.
2. Implement the chosen optimization.
3. Re-run the same benchmark script and record "after" numbers.
4. Run the crossfade and integration tests from Phases 1-3 to confirm no regression in correctness, only in speed.
5. If the optimization made things worse or made no measurable difference, document that honestly rather than discarding the attempt – this is a real and useful part of the story.

Why this matters: Companies in this space live and die by this exact kind of optimization work. Being able to say “I tried X, it improved TTFA by Y%, but cost Z in quality/complexity, so I chose to do it anyway / not do it because...” is a genuinely senior-sounding interview answer, and it only comes from actually doing the before/after measurement rather than assuming an optimization helped.

What to inspect: Your Phase 3 benchmark harness — reuse it exactly, changing nothing except the one variable under test, or your before/after comparison is meaningless.

What to learn: Where the actual bottleneck lives — model forward-pass time, codec decode time, or the crossfade/stitching logic. Profile before you optimize; don't guess.

Testing/verification: Before/after numbers in hand, all prior tests still passing.

Edge cases: Confirm the optimization doesn't silently break the mismatched-speaker discrimination test from Phase 2 (e.g. a conditioning cache reused incorrectly across different speakers would be a serious, subtle bug).

Docs to update: Add a benchmarks/optimization_log.md entry: hypothesis, change made, before/after numbers, conclusion.

Update PROJECT_STATE.md:

```
## Decisions Log
- [date] Optimization attempted: <A/B/C>. Result: <numbers>. Kept: <yes/no, why>.
```

Acceptance checklist: - [] Before/after numbers recorded for the exact same benchmark - [] All correctness tests from prior phases still pass - [] Speaker-conditioning correctness re-verified if caching was touched - [] Honest conclusion recorded even if the optimization didn't help - [] PROJECT_STATE.md updated

Phase 5 — Evaluation Harness

PROMPT 5: Build a proper, repeatable evaluation suite

Read PROJECT_STATE.md. Then build an `evaluation/` module that, given a list of (text, reference_audio) pairs, runs the full pipeline and reports:

1. TTFA and RTF (reuse Phase 3/4 instrumentation).
2. Speaker similarity (reuse Phase 2's embedding approach) – report mean and per-sample scores.
3. Intelligibility: run the generated audio back through a pretrained speech-to-text model (e.g. faster-whisper or any available ASR) and compute word error rate (WER) against the original input text. This is a standard, defensible proxy for naturalness/correctness when you don't have access to human raters.
4. Save all results as a single `evaluation/report.json` plus a short human-readable `evaluation/report.md` summarizing the numbers.
5. Write this as a script runnable via a single command, and confirm it runs end-to-end on at least 10 varied (text, speaker) pairs.

Why this matters: A demo that “sounds good to me” is not evidence. Every serious voice AI team evaluates with WER (as a naturalness/intelligibility proxy) and speaker similarity as standard metrics — building this harness yourself, rather than eyeballing outputs, is what makes your numbers in Phase 3/4 credible rather than anecdotal.

What to inspect: Make sure the evaluation pairs include some genuinely hard cases (numbers, proper nouns, longer sentences) and not just easy short phrases — an evaluation suite that only tests easy inputs will give you a false sense of quality.

What to learn: WER's limitations as a naturalness proxy — it measures whether an ASR system can transcribe your output correctly, not whether it sounds pleasant or expressive. Be ready to state this limitation explicitly rather than oversell the metric.

Testing: Run the full harness end-to-end; confirm the JSON/MD outputs are generated and numbers are sane (WER between 0 and 1, similarity between -1 and 1).

Edge cases: A generated sample that's silent or corrupted should be caught and flagged in the report, not silently skipped or crash the whole run.

Docs to update: evaluation/README.md explaining each metric, why it was chosen, and its known limitations.

Update PROJECT_STATE.md:

```
## Evaluation Results (baseline, [date])
- Mean speaker similarity: <value>
- Mean WER: <value>
- Mean TTFA / RTF: <values>
```

Acceptance checklist: - [] Harness runs end-to-end on 10+ pairs without manual intervention - [] Report files are generated and numbers are sane - [] At least 2 “hard” test cases included and their

results specifically called out - [] PROJECT_STATE.md updated with baseline numbers

Phase 6 — Robustness and Failure-Mode Hardening

PROMPT 6: Handle the failure modes you deferred earlier

Read PROJECT_STATE.md's "Known limitations" notes from every prior phase.

Compile them into a single list, then:

1. For each one, decide: fix now, or explicitly document as a known limitation with a workaround (both are legitimate outcomes – not everything needs to be fixed).
2. At minimum, ensure the pipeline does not crash (returns a clear error instead) on: empty text input, reference audio shorter than ~1 second, reference audio that is silent, and text longer than some reasonable maximum (define and enforce a limit).
3. Add tests for each of these specifically, asserting a graceful error rather than an unhandled exception.
4. Re-run the full test suite from all prior phases to confirm nothing regressed.

Why this matters: Interviewers and, more importantly, real production systems care enormously about the boundary between “works on the happy path” and “handles the world as it actually shows up,” which includes bad input. This is also cheap, high-leverage work relative to the modeling phases.

What to inspect: Every “Known limitations” note across PROJECT_STATE.md — this step is explicitly about paying down the deferred debt from earlier phases, not inventing new scope.

What to learn: The difference between a bug and a documented limitation — both are fine to have, but only if you know which is which and can say so plainly.

Testing: New edge-case tests plus full regression run of everything from Phases 1–5.

Docs to update: A single LIMITATIONS.md at the repo root listing everything you deliberately chose not to fix and why — this document itself is a strong interview artifact; it shows judgment about scope.

Update PROJECT_STATE.md:

```
## Hardening Pass ([date])  
- Fixed: <list>  
- Documented as known limitation: <list, with reasons>
```

Acceptance checklist: - [] All four minimum edge cases handled gracefully with tests - [] Full regression suite passes - [] LIMITATIONS.md exists and is honest, not defensive - [] PROJECT_STATE.md updated

Phase 7 — Integration into the Existing App

PROMPT 7: Wire the new pipeline into the existing Speechify clone's backend

Re-inspect the existing app's API layer (from Prompt 0's notes). Then:

1. Add a new endpoint (do not remove the old one yet) that accepts text plus a reference audio upload, and streams audio chunks back to the client as they're generated – use whatever streaming mechanism fits the existing stack (chunked HTTP response, Server-Sent Events, or a WebSocket if the app already uses one elsewhere; if none, pick the simplest option that supports streaming).
2. Keep the new voice-cloning modules decoupled – the endpoint should call into ``voice_clone/`` and ``audio_codec/``, not reimplement logic inline.
3. Write an integration test that hits the new endpoint with a test client, uploads a sample reference clip, and confirms it receives multiple audio chunks over time rather than one blocking response.
4. Confirm the OLD endpoint/feature still works untouched.

Why this matters: A model that only runs from a script isn't a product feature. This step is where the project stops being "a notebook that works" and becomes "a service," which is the actual job you're interviewing for.

What to inspect: The existing routing/controller layer, how the current app handles file uploads (for the reference audio), and how (if at all) it currently streams any response.

What to learn: Whatever streaming transport you pick, understand why it was the right choice for your stack rather than copying a pattern blindly — be ready to explain the tradeoff between, say, Server-Sent Events (simple, one-directional) and WebSockets (bidirectional, more setup) if asked.

Testing: New integration test plus full regression of the old endpoint's tests.

Edge cases: Client disconnects mid-stream — does the server clean up generation resources, or does it keep computing into the void? Concurrent requests — does one slow request block others?

Docs to update: API documentation (OpenAPI/README, whatever the project already uses) describing the new endpoint, its request/response shape, and its streaming behavior.

Update PROJECT_STATE.md:

```
## API Integration ([date])
- New endpoint: <path/method>
- Streaming transport: <SSE/WebSocket/chunked HTTP> – chosen because <reason>
- Old endpoint status: unchanged and passing
```

Acceptance checklist: - [] New endpoint streams multiple chunks, verified by test - [] Old endpoint's existing tests still pass unmodified - [] Client-disconnect and concurrency edge cases checked - [] PROJECT_STATE.md updated

Phase 8 — Frontend/UI Integration

PROMPT 8: Add a minimal UI for reference-audio upload and streamed playback

Re-inspect the existing frontend (from Prompt 0's notes). Then:

1. Add a UI flow: upload/record a short reference audio clip, enter text, click synthesize, and hear audio start playing as soon as the first chunk arrives (not after the full response completes) – use the Web Audio API or MediaSource Extensions, whichever fits the streaming transport chosen in Phase 7.
2. Show a visible "time to first audio" number in the UI itself – this turns your latency work into something a non-technical interviewer can literally see and hear in a live demo.
3. Keep this UI minimal – do not build extensive styling or unrelated features. This is a demo instrument, not a product.
4. Manually test in a real browser: confirm audio actually starts before the full clip has finished generating (compare the displayed TTFA to a stopwatch, or log full generation time separately to confirm the gap).

Why this matters: In an interview, “here’s a live demo where you can hear it start talking in under half a second” is dramatically more convincing than a README with numbers in it. This step converts your backend engineering into something visceral.

What to inspect: The existing frontend’s audio-playback code, if any, and its build/dev-server setup.

What to learn: How streaming audio playback actually works in a browser — this is a genuinely useful, transferable web-audio skill beyond this project.

Testing: Manual, in-browser — this step is inherently hard to fully unit test, so be rigorous about manual verification instead, and note that explicitly rather than pretending automated coverage exists where it doesn’t.

Edge cases: Slow network simulation (browser devtools throttling) — does the UI degrade gracefully rather than breaking? Very long text — does the UI stay responsive during a long generation?

Docs to update: A short demo.md or GIF/screen-recording reference showing the live latency number.

Update PROJECT_STATE.md:

```
## UI Integration ([date])  
- Confirmed manually: audio begins playback before full generation completes
```

- Displayed TTFA in UI matches backend-logged TTFA within <tolerance>

Acceptance checklist: - [] Live demo confirmed working in-browser by you personally - [] TTFA is visibly displayed and matches backend logs - [] Graceful behavior under throttled network, checked manually - [] PROJECT_STATE.md updated

Phase 9 — Monitoring and Observability

PROMPT 9: Add structured logging and a simple metrics view

Read PROJECT_STATE.md. Then:

1. Add structured logging (not print statements) around the synthesis
 endpoint: log request start/end, TTFA, RTF, speaker-similarity (if computed at request time), and any errors, all with a request ID.
2. Persist these logs to a simple store – a local SQLite table or an append-only JSON-lines file is enough; do not set up a heavy monitoring stack for a portfolio project.
3. Build one small script or page that reads recent logs and shows:
 a rolling average TTFA and RTF over the last N requests, and error rate.
4. Simulate 20 varied requests (different text lengths, at least one deliberately bad input from Phase 6) and confirm the dashboard/script reflects them accurately, including the error from the bad input.

Why this matters: Voice AI is a latency-sensitive, production service, not a one-off script — showing that you think about observability (not just “does it work once”) signals SWE/production maturity that pure ML-focused candidates often lack, and it’s a specific, concrete thing you can point to that most portfolio projects skip entirely.

What to inspect: Whatever logging convention the existing app already uses — extend it rather than introducing a second, inconsistent logging system.

What to learn: The difference between a log line and a metric, and why aggregating many requests into rolling averages is more useful for spotting regressions than reading logs one at a time.

Testing: Run the 20-request simulation, confirm the aggregate view matches what you’d compute by hand from the raw logs for a small sample.

Edge cases: Confirm the error case from Phase 6 shows up correctly in the error-rate metric rather than being silently dropped from the logs.

Docs to update: MONITORING.md explaining what’s logged and how to view the rolling metrics.

Update PROJECT_STATE.md:

```
## Observability ([date])  
- Logging: <mechanism>  
- Verified via 20-request simulation, including 1 induced failure, correctly reflected in error rate
```

Acceptance checklist: - [] Structured logs captured for all requests including request ID - [] Rolling TTFA/RTF/error-rate view works and was spot-checked by hand - [] Induced failure correctly shows up in error metrics - [] PROJECT_STATE.md updated

Phase 10 — Deployment and Final Packaging

PROMPT 10: Containerize and produce the final interview-ready artifact

Read the entire PROJECT_STATE.md file for full project history.
Then:

1. Write a Dockerfile (or update the existing one) that runs the full app including the new voice-cloning pipeline, and confirm it builds and runs locally end-to-end (upload reference audio, get streamed cloned speech back) from a clean container.
2. Write a top-level README.md section (or new ARCHITECTURE.md) that explains, in your own words: the overall pipeline (text + reference audio -> codec/model -> streaming chunks -> playback), the key engineering decisions and their tradeoffs (pulling directly from the Decisions Log in PROJECT_STATE.md), your final evaluation numbers (from Phase 5, plus before/after from Phase 4), and explicitly stated limitations (from LIMITATIONS.md).
3. Do a final full regression run of every test written across all 10 phases and confirm everything passes in the containerized environment, not just your local dev environment.
4. Write a short "why this problem matters" paragraph connecting the project back to real products (ElevenLabs, Speechify, conversational voice agents generally) for your own reference before interviews.

Why this matters: This is the step that turns ten phases of work into a single, coherent artifact someone can clone, run, and understand in fifteen minutes — and that you can talk through confidently for the same fifteen minutes in an interview.

What to inspect: The full PROJECT_STATE.md history — this is the moment it pays off as your source of truth for the write-up, rather than trying to remember what you decided three weeks ago.

What to learn: Being able to summarize ten phases of engineering decisions concisely, with tradeoffs stated plainly, is itself a skill worth practicing — it's close to exactly what a technical interview conversation will demand of you.

Testing: Full test suite inside the container, clean build from scratch (no cached local state).

Edge cases: Confirm the containerized version handles the same failure modes from Phase 6 correctly — container environments sometimes expose bugs (missing system libraries, different default configs) that don't show up locally.

Docs to update: README.md/ARCHITECTURE.md as the final, polished deliverable; LIMITATIONS.md reviewed one more time for accuracy.

Update PROJECT_STATE.md:

```
## Final Status ([date])  
- Full regression suite passing in container: yes/no  
- README/ARCHITECTURE.md finalized  
- Ready for portfolio/interview use
```

Final acceptance checklist for the whole project: - [] Container builds clean and runs the full pipeline end-to-end - [] All tests from Phases 1-9 pass inside the container - [] README/ARCHITECTURE.md clearly explains pipeline, decisions, tradeoffs, numbers, and limitations - [] You can, without notes, explain: what a neural audio codec does, how voice conditioning works, why chunked streaming reduces latency, what your TTFA/RTF/WER/speaker-similarity numbers mean, and at least one optimization you tried and its actual measured effect

A note on pacing

Ten phases is a real project, not a weekend one. If you need to timebox it, Phases 0-3 are the non-negotiable core that demonstrates the central technical idea (codec representation, voice conditioning, streaming). Phases 4-6 are what make the numbers credible rather than anecdotal. Phases 7-10 are what make it a demonstrable product instead of a script — valuable, but if you're ever forced to cut scope under time pressure, cut from the end, not the middle.